

Protecting System Intents in Android

The last article on 'Intents in Android' covered the basics of one of Android's important inter-application communication mechanisms—intents. This article goes deeper into Android's System Intents, and their security aspects, i.e., how Android prevents applications from misusing system-level intents.



System-level intents are those that the Android System sends—a more appropriate term would be 'broadcasts'—to the rest of the system and the applications, to notify them about a certain event. These are mostly sent by various services that are part of the core Android framework. These intent broadcasts can be *sticky*, *ordered*, etc, depending on the nature of the event and the receivers. On receiving these intents, the registered listeners take necessary action.

Why do system intents need to be protected?

A normal intent is something like `'ACTION_PHONE_CALL'`, which an application can send (provided it has the required permission granted during installation) to initiate a phone call; whereas, an easy example for a malicious system intent would be `'ACTION_SHUTDOWN'`. No application should be able to shut down your device. Another example would be `'ACTION_BATTERY_CHANGED'`, which is broadcast when the phone's battery properties are changed. The system application that shows the battery capacity receives this intent,

extracts the battery properties (one of which is the capacity) and shows it in the notification area. If a malicious application were able to send this intent, it might send erroneous values for the capacity, and as a result, the end user might see strange results in how the battery capacity is reported.

Protecting system-level intents

When an application sends an intent, the Activity Manager, which plays a major role in managing the life-cycle of an application and in Inter-Process Communication (IPC), verifies the permissions of the sender for the particular intent. If an application sends a system-level intent, this permission check will fail and an exception will be thrown.

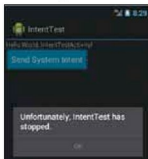


Figure 1: App has stopped